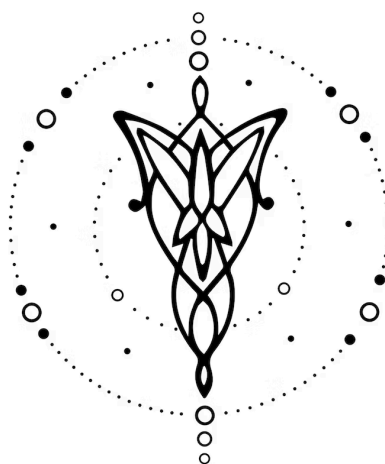




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

5 de Abril de 2026

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	3
6. Ejemplos	4

1. Equipo

Nombre	Apellido	Legajo	E-mail
Valentin	Ruiz	64046	vruiz@itba.edu.ar
Santiago	Allegretti	65531	sallegretti@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/vruizitba/Flex-Bison-Compiler>

3. Dominio

El lenguaje propuesto se enmarca en el dominio de la programación de sistemas con abstracciones orientadas a objetos, con el objetivo de facilitar la construcción de programas estructurados sobre una base de ejecución en C. Para ello, incorpora mecanismos que permiten modelar entidades mediante clases que agrupan estado y comportamiento, abstrayendo limitaciones propias de C, donde estas estructuras deben resolverse manualmente a través de structs y funciones asociadas.

En este dominio, los programas se conciben como un conjunto de objetos que interactúan entre sí mediante invocaciones de métodos, manteniendo su estado interno controlado y favoreciendo una organización más clara del código. De esta manera, el lenguaje apunta a mejorar la claridad, la reutilización y la separación de responsabilidades en el desarrollo de software.

El lenguaje está diseñado con una sintaxis inspirada en C, pero extendida con construcciones orientadas a objetos, tales como clases, atributos, métodos y control de visibilidad. Los programas escritos en este lenguaje son posteriormente traducidos a código C equivalente, permitiendo su ejecución eficiente sin requerir cambios en el entorno de compilación subyacente.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán definir clases, cada una con un identificador único.
- (II). Las clases podrán contener atributos y métodos.
- (III). Se podrán definir atributos de tipo primitivo (int, bool, string, etc.) o de tipo objeto.
- (IV). Se podrán definir métodos con o sin parámetros, y con tipo de retorno o void.
- (V). Se permitirá el uso de la instrucción “return” para devolver valores desde los métodos.
- (VI). Se permitirá definir modificadores de visibilidad (public, private, protected) para atributos y métodos, cuya validez será verificada en tiempo de compilación.
- (VII). Se podrá utilizar la palabra reservada “this” para referenciar la instancia actual dentro de los métodos.
- (VIII). Se permitirá la instanciación de objetos mediante una construcción específica (new).
- (IX). Se podrán definir constructores para inicializar objetos al momento de su creación, los cuales podrán ser invocados mediante la construcción “new”.
- (X). Se podrá acceder a atributos y métodos mediante el operador “.” respetando las reglas de visibilidad definidas.
- (XI). Se podrán declarar variables tanto de tipo primitivo como de tipo clase, así como realizar asignaciones sobre las mismas.
- (XII). Se proveerán estructuras de control de flujo tales como if, if-else, while y for.
- (XIII). El lenguaje soportará expresiones aritméticas, relacionales y lógicas, incluyendo operadores como +, -, *, /, ==, !=, <, >, &&, || y !.

- (XIV). Se definirá un punto de entrada al programa mediante un bloque principal (main).
- (XV). Se podrán definir métodos y atributos estáticos asociados a la clase.
- (XVI). Se podrá definir herencia simple entre clases mediante la palabra reservada "extends". Dado que el lenguaje no contempla una construcción equivalente a "super" (Java), no se garantiza la inicialización explícita de los atributos heredados a través del constructor de la superclase.
- (XVII). Se permitirá la sobrecarga de métodos, pudiendo definir múltiples métodos con el mismo nombre dentro de una clase, diferenciados por la cantidad y/o tipo de parámetros, cuya validez será verificada en tiempo de compilación.

5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que define una clase con constructor y crea una instancia pasando parámetros para su inicialización.
- (II). Un programa que define una clase con atributos públicos y accede a dichos atributos desde el bloque principal.
- (III). Un programa que define una clase con atributos privados y permite acceder a ellos mediante métodos públicos definidos en la misma clase.
- (IV). Un programa que define una clase y crea múltiples instancias de la misma.
- (V). Un programa que define una clase y otra que hereda de ella mediante la palabra reservada extends.
- (VI). Un programa que define un método estático y lo invoca sin necesidad de instanciar la clase.
- (VII). Un programa que define métodos con parámetros y retorno, y utiliza los valores retornados en expresiones.
- (VIII). Un programa que define múltiples métodos con el mismo nombre pero distintos parámetros, y realiza llamadas válidas a cada uno de ellos.
- (IX). Un programa que utiliza la palabra reservada "this" dentro de un método para acceder y modificar atributos de la instancia.
- (X). Un programa que declara variables de tipo primitivo y de tipo clase, realizando asignaciones entre valores compatibles.
- (XI). Un programa que utiliza estructuras de control (if, if-else, while, for) dentro de los métodos de una clase.
- (XII). Un programa que define una clase hija que accede a métodos públicos heredados de la clase padre.
- (XIII). Un programa que define una clase padre con atributos o métodos protected, y una clase hija que accede a dichos miembros heredados desde uno de sus propios métodos.

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa que intenta instanciar una clase que no fue definida previamente.

- (II). Un programa que accede directamente a un atributo definido como privado desde fuera de la clase.
- (III). Un programa que invoca un método que no se encuentra definido en la clase correspondiente.
- (IV). Un programa que realiza una asignación entre variables de tipos incompatibles.
- (V). Un programa que invoca un método con una cantidad o tipo de parámetros diferente al definido en su declaración.
- (VI). Un programa que define una clase que hereda de otra que no fue previamente definida.
- (VII). Un programa que invoca un método sobrecargado con parámetros que no coinciden con ninguna de sus definiciones.
- (VIII). Un programa que utiliza la palabra reservada “this” fuera del contexto de una instancia.

6. Ejemplos

Definición de una clase con atributos de distintas visibilidades (private, protected, public), junto con un constructor, métodos de acceso (getters/setters) y un método protected.

```
class Animal {  
    private string nombre;  
    protected int edad;  
    public string especie;  
  
    Animal(string n, int e, string esp) {  
        this.nombre = n;  
        this.edad = e;  
        this.especie = esp;  
    }  
  
    public string getNombre() {  
        return this.nombre;  
    }  
  
    public void setNombre(string n) {  
        this.nombre = n;  
    }  
  
    protected void crecer() {  
        this.edad = this.edad + 1;  
    }  
}
```

Definición de una clase que hereda de otra mediante extends, accediendo a atributos y métodos protected de la clase padre y utilizando métodos públicos heredados.

```
class Perro extends Animal {  
    public string raza;  
  
    Perro(string n, int e, string r) {  
        this.setNombre(n);  
        this.edad = e;  
        this.especie = "Perro";  
        this.raza = r;  
    }  
  
    public void cumplirAnios() {  
        this.crecer();  
    }  
}
```

Clase que contiene un método estático

```
class Util {  
  
    // Método estático  
    public static int suma(int a, int b) {  
        return a + b;  
    }  
}
```

Creación de instancias de las clases previamente definidas, invocación de sus métodos desde el bloque principal del programa y uso de método estático.

```
main {  
    Animal a;  
    Perro p;  
  
    a = new Animal("Tigre", 5, "Felino");  
    p = new Perro("Firulais", 3, "Labrador");  
  
    a.setNombre("Leon");  
    p.cumplirAnios();  
  
    int resultado  
    // Llamada a método estático sin crear objeto  
    resultado = Util.suma(2, 3);  
  
}
```